

Islamic University of Technology

Lab 2: Empirical Analysis of Randomized vs. Deterministic Algorithms.

CSE 4810: Algorithm Engineering
2A

ASH, ABT(PT)

August 6, 2026

General Instructions: Empirical Analysis

To observe how different randomization strategies affect the runtime and efficiency of deterministic algorithms.

1. **Implementation:** Implement the two versions (Deterministic and Randomized) of the algorithms specified in your assigned set.
2. **Experimental Setup:** Run both versions of algorithms on Random data and worst case input data. You need to generate the input data.
3. **Data Recording:** Run both algorithms over an increasing series of input sizes (n). Save both the comparison $counts(X)$ and running $time(T)$. Save your results to a text file (`data.txt`) in three columns: `n`, `Time_DM`, `Time_Rand`, `Count_DM`, `Count_Rand`.
4. **Instrumentation:** Use high-resolution timers (e.g., `std::chrono` in C++, `System.nanoTime()` in Java, or `time.perf_counter()` in Python) to measure execution time. Use count variable on appropriate places for comparison count.
5. **Visualization:** Use `pyplot` to plot your data.
6. **Theoretical Analysis:** Do ratio test for both version and expected number of comparison from probabilistic analysis.

Set 1: Extreme Pivots

- **Sorting Task:** Compare Deterministic Quicksort (`Pivot = A[low]`) with Randomized Quicksort (Uniformly random pivot).
- **Selection Task:** Compare Quickselect with a fixed first-element pivot vs. Randomized-Select for any given k .
- **Engineering Goal:** Plot execution time and comparison count versus n . Prepare to explain to the instructors the reasoning behind the observed behavior of the algorithms.

Student Manual: High-Resolution Timing and Python Pyplot

This manual provides the standard operating procedures for timing your code in C++, Java, and Python, followed by instructions for plotting your results using Python's `matplotlib.pyplot` library.

Part 1: Instrumentation (Time Calculation)

CRITICAL RULE: Never place I/O operations (like `printf`, `cout`, or `print`) inside your timing blocks. Console output is thousands of times slower than basic arithmetic and will completely corrupt your empirical data.

1. C++ (Using `std::chrono`)

Do not use the legacy C-style `clock()` function, as it can be inaccurate on modern multi-core systems. Use the C++11 `chrono` library for high-resolution wall-clock time.

```
#include <bits/stdc++.h>
#include <chrono>

using namespace std;
using namespace std::chrono;

// Setup the RNG Engine
std::random_device rd;
std::mt19937 gen(rd());

// 1. Uniform RNG between [min, max]
int getRandom(int min, int max) {
    std::uniform_int_distribution<> distr(min, max);
    return distr(gen);
}

// 2. Random Shuffle
void shuffleArray(std::vector<int>& arr) {
    std::shuffle(arr.begin(), arr.end(), gen);
}

int main() {
    int n = 1000;
    // ... initialize your array/data here ...

    // Start timer
    auto start = high_resolution_clock::now();

    // Execute ONLY the algorithm
    myAlgorithm(arr, n);

    // Stop timer
```

```

    auto stop = high_resolution_clock::now();

    // Calculate duration in microseconds
    auto duration = duration_cast<microseconds>(stop - start);

    cout << "Time taken: " << duration.count() << " microseconds" << endl;

    return 0;
}

```

2. Java (Using `System.nanoTime()`)

Avoid `System.currentTimeMillis()`, as its resolution is often tied to the operating system's time-slicing granularity.

```

import java.util.Random;
import java.util.concurrent.ThreadLocalRandom;

public class LabUtils {
    // 1. Uniform RNG between [min, max]
    public static int getRandom(int min, int max) {
        return ThreadLocalRandom.current().nextInt(min, max + 1);
    }

    // 2. Random Shuffle (Fisher-Yates)
    public static void shuffleArray(int[] arr) {
        Random rnd = ThreadLocalRandom.current();
        for (int i = arr.length - 1; i > 0; i--) {
            int index = rnd.nextInt(i + 1);
            // Simple swap
            int temp = arr[index];
            arr[index] = arr[i];
            arr[i] = temp;
        }
    }
}

public class Profiler {
    public static void main(String[] args) {
        int n = 1000;
        // ... initialize your array/data here ...

        // Start timer
        long startTime = System.nanoTime();

        // Execute ONLY the algorithm
        myAlgorithm(arr, n);

        // Stop timer
        long endTime = System.nanoTime();

        // Calculate duration in microseconds
    }
}

```

```

        long duration = (endTime - startTime) / 1000;

        System.out.println("Time taken: " + duration + " microseconds");
    }
}

```

3. Python (Using `time.perf_counter()`)

Do not use `time.time()`. Use `time.perf_counter()` which provides the highest available resolution clock for short durations.

```

import time
import random

# 1. Uniform RNG between [min, max]
# Note: randint is inclusive of both endpoints
val = random.randint(min, max)

# 2. Random Shuffle
my_list = [1, 2, 3, 4, 5]
random.shuffle(my_list) # Modifies the list directly

n = 1000
# ... initialize your array/data here ...

# Start timer
start_time = time.perf_counter()

# Execute ONLY the algorithm
my_algorithm(arr, n)

# Stop timer
end_time = time.perf_counter()

# Calculate duration in microseconds
duration = (end_time - start_time) * 1_000_000

print(f"Time taken: {duration:.2f} microseconds")

```

Part 2: Data Formatting

Your program should run your algorithms inside a loop that gradually increases n (e.g., $n = 10, 50, 100, 500, 1000 \dots$). Instead of printing to the console, write the results directly to a text file named `data.txt`. Ensure the data is formatted in 5 distinct columns separated by spaces.

Example `data.txt`:

#n	Time_DM	Time_Rand	Count_DM	Count_Rand
100	10	16	4950	595
200	43	15	19900	1398
300	93	23	44850	2408

400	158	38	79800	3543
500	243	46	124750	4859
600	352	60	179700	6019
700	470	60	244650	6804
800	614	75	319600	8338
900	777	88	404550	9282
1000	950	90	499500	10671

Part 3: Plotting with Python Pyplot

Once you have generated your `data.txt` file, you will use Python and the `matplotlib` library to visualize your asymptotic curves and find the crossover point.

1. Prerequisites

If your Ubuntu lab PC does not already have `matplotlib` installed, open your terminal and install it using the system package manager:

```
sudo apt update
sudo apt install python3-matplotlib
```

2. The Plotter Script

Create a new file named `plotter.py` in the same directory as your `data.txt` file. Copy and paste the following Python code into it:

```
import matplotlib.pyplot as plt

# Initialize lists for 5 columns
n_vals, time_dm, time_rand, count_dm, count_rand = [], [], [], [], []

# Read data from the text file
with open('data.txt', 'r') as f:
    for line in f:
        if line.startswith('#') or not line.strip():
            continue
        parts = line.split()
        n_vals.append(float(parts[0]))
        time_dm.append(float(parts[1]))
        time_rand.append(float(parts[2]))
        count_dm.append(float(parts[3]))
        count_rand.append(float(parts[4]))

# Create a figure with two subplots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 10))

# Subplot 1: Execution Time
ax1.plot(n_vals, time_dm, label='Time_DM', marker='o', color='red')
ax1.plot(n_vals, time_rand, label='Time_Rand', marker='x', color='blue')
```

```

ax1.set_title('Execution Time Comparison')
ax1.set_ylabel('Time (microseconds)')
ax1.legend()
ax1.grid(True)

# Subplot 2: Comparison Counts
ax2.plot(n_vals, count_dm, label='Count_DM', marker='s', color='green')
ax2.plot(n_vals, count_rand, label='Count_Rand', marker='^', color='orange')
ax2.set_title('Comparison Count Comparison')
ax2.set_xlabel('Input Size (n)')
ax2.set_ylabel('Number of Comparisons')
ax2.legend()
ax2.grid(True)

plt.tight_layout()
plt.show()

```

3. Running the Script & Finding the Crossover Point

Run the script from your terminal:

```
python3 plotter.py
```

When the interactive Pyplot window opens:

- Click the **Magnifying Glass icon** (Zoom to rectangle).
- Click and drag a box over the intersection of the two lines (usually very close to the bottom-left corner of the graph) to zoom in.
- Click the **Home icon** to reset the view to the original scale.